CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Test Document

for

AudioScholar: Transforming Audio into Actionable Insights for Learners

AudioScholar: Transforming Audio into Actionable Insights for Learners

Proponent(s):

Biacolo, Math Lee L.
Terence, John Duterte
Orlanes, John Nathan
Barrientos, Claive Justin
Alpez, Christian Brent

Adviser:

Jasmine A. Tulin

_____

Consultation Schedule:

TULIWED11001200

Date of Submission:

September 20, 2025

Change History

VERSION	DATE	AUTHOR	DESCRIPTION OF CHANGE
1.0	2025-19-09	BIACOLO	Initial Draft
2.0	2025-22-10	ORLANES	Second Draft
3.0	2025-28-10	BIACOLO	Comprehensive revision based on existing documents.
4.0	2025-01-11	BARRIENTOS	Appendix
5.0	2025-10-11	ORLANES	Appendix
6.0	2025-13-11	ORLANES	Finalization
6.1	2025-14-11	BIACOLO	Verified & Finalized

Table of Contents

Change History **3**
Table of Contents **4**
1. Introduction **5**
1.1 System Overview 5
1.2 Test Approach 6
1.2.1 Levels of Testing *6*
1.2.2 Types of Testing *7*
1.3 Definitions and Acronyms 8
2. Test Plan **11**
2.1 Features to be Tested 11
2.2 Features not to be Tested 12
2.3 Testing Tools and Environments 13
2.4 Test Schedule and Resources 14
2.5 Entry and Exit Criteria 15
2.6 Defect Management 15
3. Test Cases **17**
3.1 Case-1: Successful Offline Lecture Recording and Local Save 17
3.1.1 Purpose *17*
3.1.2 Inputs *17*
3.1.3 Expected Outputs & Pass/Fail Criteria *17*
3.1.4 Test Procedure *18*
3.2 Case-2: Web Upload with File Exceeding Size Limit 19
3.2.1 Purpose *19*
3.2.2 Inputs *19*
3.2.3 Expected Outputs & Pass/Fail Criteria *19*
3.2.4 Test Procedure *20*
3.3 Case-3: End-to-End Audio Summarization Workflow 21
3.3.1 Purpose *21*
3.3.2 Inputs *21*
3.3.3 Expected Outputs & Pass/Fail Criteria *21*
3.3.4 Test Procedure *22*
3.4 Case-4: User Login with Invalid Password 23
3.4.1 Purpose *23*
3.4.2 Inputs *23*
3.4.3 Expected Outputs & Pass/Fail Criteria *23*
3.4.4 Test Procedure *24*
4. Appendix (Test Logs) **33**
A.1 Test Log Template 33
A.2 Test Summary Report Template 36
A.3 Incident Report Template 37

1. Introduction

1.1 System Overview

AudioScholar is an educational technology (EdTech) platform designed to enhance the learning process by converting lecture audio into actionable study materials. The system operates on a client-server architecture, comprising an Android mobile application for on-the-go recording and a ReactJS web interface for comprehensive content management. The backend, built with Spring Boot, orchestrates core functionalities including audio processing, user management, and integration with external services.

The primary value proposition is to reduce the cognitive load of manual note-taking, allowing students to focus on active listening during lectures. Post-lecture, the system provides AI-generated summaries, key points, and supplementary learning recommendations, transforming passive audio recordings into a powerful study tool. The system is designed for a multi-user environment with data privacy and implements a freemium model to manage feature access.

This document establishes the formal plan and methodology for the verification and validation of the AudioScholar system against the requirements detailed in the Software Requirements Specification (SRS v2.1).

1.2 Test Approach

The testing strategy for AudioScholar is requirements-driven and risk-based, integrated within the project's Agile/Scrum development lifecycle. The goal is to ensure that all functional and non-functional requirements are met, critical functionalities are stable, and the application provides a high-quality user experience.

1.2.1 Levels of Testing

1. **Unit Testing:** Performed by developers to verify individual functions, methods, and classes. This includes testing Kotlin classes in the Android app and Java services/controllers in the Spring Boot backend to ensure they behave as expected in isolation.
2. **Integration Testing:** Focuses on verifying the interaction between different components of the system. Key integration points to be tested include:
   - Mobile/Web client API calls to the backend REST endpoints.
   - Backend service communication with the Firebase/Nhost infrastructure (Authentication, Firestore, Storage).
   - Backend integration with external APIs (Google Gemini AI, YouTube Data API).
3. **System Testing:** The fully integrated application is tested end-to-end to validate compliance with all requirements specified in the SRS. This involves executing test cases based on user stories and use cases, covering the complete workflow from audio capture to summary review.
4. **User Acceptance Testing (UAT):** Conducted by a representative group of target users (college students) to confirm that AudioScholar meets their needs and expectations. This phase focuses on usability, workflow efficiency, and overall satisfaction. Feedback will be gathered through direct

observation and structured surveys.

1.2.2 Types of Testing

- **Functional Testing:** Verifies that each feature of the application works according to its specified requirements. This forms the bulk of the test cases in Section 3.
- **Non-Functional Testing:**
  - **Performance Testing:** Measures the system's responsiveness and latency against defined benchmarks (e.g., summarization time for a 30-minute audio file as per SRS 3.3).
  - **Security Testing:** Verifies the implementation of security measures, including secure data transmission (HTTPS), protection against unauthorized access via Firebase rules, and secure management of API keys.
  - **Usability Testing:** Assesses the intuitiveness and ease of use of the mobile and web interfaces during UAT.
  - **Compatibility Testing:** Ensures the web interface functions correctly on modern browsers (Chrome, Firefox) and the mobile app runs on target Android versions (Android 7.0+).

1.3 Definitions and Acronyms

Term/Acronym	Definition
AI	**Artificial Intelligence:** The field of computer science dedicated to creating systems that perform tasks requiring human intelligence. In AudioScholar, this refers to the use of the Google Gemini AI API for audio summarization.
API	**Application Programming Interface:** A set of rules and protocols that allows different software applications to communicate with each other.
ERD	**Entity Relationship Diagram:** A diagram that illustrates the conceptual structure of a database, showing entities and their relationships.
Firebase	A mobile and web application development platform by Google, used in AudioScholar for Authentication and the Firestore database.
Freemium	A business model where basic services are provided free of charge, while advanced features are offered for a premium fee.
Git	A distributed version control system used for tracking changes in source code during software development.
GitHub	A web-based hosting service for Git version control, used for project source code management.
Google Gemini AI	Google's advanced AI API suite used by AudioScholar for audio processing and summarization.
HTTPS	**Hypertext Transfer Protocol Secure:** The secure version of HTTP, which encrypts communication between clients and servers.

IDE	**Integrated Development Environment:** A software application that provides comprehensive facilities to computer programmers for software development (e.g., Android Studio, VS Code).
Kotlin	A modern, statically-typed programming language used for developing the AudioScholar Android application.
NFR	**Non-Functional Requirement:** A requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors (e.g., performance, security).
Nhost	A backend development platform used in AudioScholar for cloud file storage.
OAuth 2.0	An open standard for access delegation, commonly used to grant websites or applications access to user information on other websites without giving them the passwords.
ReactJS	A JavaScript library for building user interfaces, used to develop the AudioScholar web interface.
Render	A cloud application hosting service used to deploy the AudioScholar backend.
REST	**Representational State Transfer:** An architectural style for designing networked applications, used for the backend APIs.
RRF	**Requirements Reference File:** The document that lists and details all project goals and their associated requirements.
SDD	**Software Design Description:** The document that translates software requirements into a blueprint for implementation, detailing the system's architecture and design.

SPMP	**Software Project Management Plan:** The document that outlines the processes and plans for managing the project.
Spring Boot	An open-source Java-based framework used to create microservices and web applications, used for the AudioScholar backend.
SRS	**Software Requirements Specification:** The document that describes the functional and non-functional requirements of the software to be developed.
STD	**Software Test Document:** This document, which outlines the scope, approach, resources, and schedule of intended testing activities.
TC	**Test Case:** A set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly.
UAT	**User Acceptance Testing:** The final phase of testing where end-users validate the software against their requirements.
UI	**User Interface:** The visual and interactive elements through which a user interacts with a software application.
UX	**User Experience:** The overall experience a user has when interacting with a product, especially in terms of how easy or pleasing it is to use.

2. Test Plan

2.1 Features to be Tested

- **G1: Efficient Lecture Audio Recording and Management**
  - **REQ-101:** Offline lecture recording on the mobile app.
  - **REQ-102:** Upload of pre-recorded audio files from both mobile and web platforms, including validation of file size (≤ 500MB) and supported formats.
- **G2: AI-Driven Summary and Recommendation Generation**
  - **REQ-201:** Automated, server-side audio processing and generation of structured summaries via the Gemini AI API.
  - **REQ-202:** Analysis of summary content and generation of relevant YouTube video recommendations.
  - **REQ-203:** Optional upload and association of PowerPoint files to enhance summarization context.
- **G3: Secure, Multi-User Environment with Cloud Sync**
  - **REQ-301:** Secure user registration and login flows (Email/Password, Google OAuth, GitHub OAuth).
  - **REQ-302:** Optional cloud synchronization of user data between the mobile client and Nhost cloud storage.

- **REQ-303:** Verification of end-to-end HTTPS encryption for all client-server communication.

- **G4: Freemium Model Implementation**
  - **REQ-401:** Role-based access control restricting features based on user subscription tier (Free vs. Premium).
  - **REQ-402:** Simulated payment workflow for upgrading from a free to a premium account.

2.2 Features not to be Tested

To maintain focus and adhere to the project scope, the following features are explicitly excluded from testing in this version.

Feature	Rationale for Exclusion
iOS Mobile Platform Support	The initial development is restricted to the Android platform as per SRS Section 1.2.
Real-time Transcription	This feature is not a planned requirement for version 1.0. The focus is on post-lecture summarization.
Web Interface Audio Recording	The web interface is designed for content management and upload only; direct recording is out of scope.
Multi-language Support	The system is designed to support only English language lectures initially.

2.3 Testing Tools and Environments

Category	Tool/Technology	Purpose
Test Management	ClickUp	Task management, informal defect tracking, and progress monitoring.
Version Control	Git / GitHub	Source code management and branching for test builds.
Mobile IDE	Android Studio	Development, debugging, and execution of mobile unit tests.
Web/Backend IDE	Visual Studio Code	Development and debugging of web and backend components.
API Testing	Postman / Insomnia	Manual testing of backend REST API endpoints.
Client Environment	Android Devices (Android 7.0+), Google Chrome, Mozilla Firefox	Execution of system and UAT test cases.
Backend Environment	Render (Cloud Host), Java (Spring Boot)	Production and staging environment for the backend application.
Data Environment	Firebase (Firestore, Authentication), Nhost (Storage)	Data persistence, user management, and file storage.

2.4 Test Schedule and Resources

Testing activities are integrated into the 8-week Agile/Scrum development timeline.

- **Weeks 1-7:** Continuous unit and integration testing performed by developers alongside feature development.
- **Weeks 6-7:** Dedicated system testing phase executed by the entire team to validate end-to-end workflows.
- **Week 8:** User Acceptance Testing (UAT) with target student users, followed by final regression testing.

Resources: All five team members will participate in testing activities, with roles assigned based on their development focus (mobile, web, backend). The Team Leader will coordinate the overall testing effort.

2.5 Entry and Exit Criteria

- **Entry Criteria (for System Testing):**
  - All major features have been developed and have passed unit and integration tests.
  - A stable build is deployed to the testing environment.
  - The test plan and test cases have been reviewed and approved.
- **Exit Criteria (for Release):**
  - All planned test cases have been executed.
  - A pass rate of 98% or higher for all test cases.
  - No outstanding “Critical” or “High” severity defects.
  - All UAT feedback has been reviewed, and critical issues have been addressed.

2.6 Defect Management

Defects identified during testing will be managed as follows:

1. **Reporting:** Defects will be reported directly to the relevant team member(s) and logged as tasks/subtasks in ClickUp.
2. **Prioritization:** The Team Leader will prioritize defects based on severity (Critical, High, Medium, Low) and impact on core functionality.

3. **Resolution:** Developers will fix prioritized defects. Fixes will be deployed to the test environment for verification.

4. **Verification & Closure:** The tester who reported the defect will re-test the functionality to verify the fix. Upon successful verification, the defect task in ClickUp will be closed.

3. Test Cases

3.1 Case-1: Successful Offline Lecture Recording and Local Save

3.1.1 Purpose

To verify that a user can successfully start, stop, and save an audio recording on the Android application without an active internet connection, ensuring the file is correctly stored in the device's local storage and is accessible within the app. This test validates the core offline capability outlined in REQ-101.

3.1.2 Inputs

- **Hardware:** Android device (Android 7.0+) with a functional microphone and a minimum of 50MB of available local storage.
- **Software/System State:** The AudioScholar mobile application is installed. The user is authenticated and logged into the application. The application has been granted microphone and storage permissions.
- **Configuration:** The device's network connectivity is disabled (e.g., Airplane Mode is ON).
- **User Data:** N/A.

3.1.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** Upon tapping the 'Record' button, the UI must provide clear visual feedback that recording has commenced (e.g., a running timer). Upon tapping 'Stop', a dialog should prompt the user to provide a title. After saving, a confirmation message should be displayed, and the new recording should appear in the app's local library list.

- **Pass Criteria:** The test is considered 'Pass' if the audio file is successfully recorded, saved with the user-provided title to the device's local storage, and is present and playable from within the application's library view.
- **Fail Criteria:** The test is considered 'Fail' if the application crashes, fails to initiate or terminate recording, displays an error during the save process, the file is not found in the local library, or the saved file is corrupted and unplayable.

3.1.4 Test Procedure

---

1. Verify the test device has sufficient storage and is in Airplane Mode.
2. Launch the AudioScholar application.
3. Navigate to the 'Record' screen.
4. Tap the primary record button to begin recording.
5. Confirm the UI updates to show a running timer and a "recording" status indicator.
6. Allow the recording to run for at least 20 seconds while speaking into the microphone.
7. Tap the 'Stop' button to terminate the recording.
8. In the prompt that appears, enter the title: `Offline Recording Test 01`.
9. Tap the 'Save' button.
10. Verify a confirmation message (e.g., a Snackbar or Toast stating "Recording saved successfully") is displayed.

11. Navigate to the 'Library' or 'My Recordings' section of the app, ensuring the view is filtered for local files.

12. Verify that a recording titled `Offline Recording Test 01` is present in the list.

3.2 Case-2: Web Upload with File Exceeding Size Limit

3.2.1 Purpose

To verify that the web interface correctly rejects an audio file that exceeds the maximum allowed size limit of 500MB, as specified in REQ-102, and provides clear, immediate feedback to the user without attempting to upload the file.

3.2.2 Inputs

- **Hardware:** A workstation with a modern web browser (e.g., Google Chrome, Mozilla Firefox).
- **Software/System State:** The user is logged into the AudioScholar web interface.
- **Configuration:** A stable internet connection is active.
- **Test Data:** A single audio file (e.g., .mp3, .wav) with a size definitively greater than 500MB (e.g., 525MB).

3.2.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** The system should display an error message on the UI immediately after the file is selected or when the upload is attempted. The file upload process should not begin.

- **Pass Criteria:** The test passes if a clear error message (e.g., "File size exceeds the 500MB limit.") is displayed to the user, and no network request to upload the file is initiated (as verifiable in the browser's developer tools).

- **Fail Criteria:** The test fails if the upload process begins, the browser or application freezes or crashes, no error message is shown, or a generic, non-descriptive error message (e.g., "Upload failed") is displayed.

3.2.4 Test Procedure

---

1. Using a compatible web browser, log into the AudioScholar web application.
2. Navigate to the 'Upload Audio' page.
3. Click the file selection control (e.g., "Choose File" button).
4. Select the test audio file that is larger than 500MB from the local file system.
5. Observe the UI for any immediate validation messages.
6. If no immediate message appears, click the final 'Upload' button.
7. Verify that the specified error message is displayed on the screen.
8. (Optional) Open the browser's developer tools, switch to the 'Network' tab, and confirm that no large file upload request was sent.

3.3 Case-3: End-to-End Audio Summarization Workflow

3.3.1 Purpose

To verify the complete, end-to-end system functionality of uploading an audio file, processing it via the backend AI service, and successfully retrieving and displaying the generated summary, validating the core workflow of REQ-201.

3.3.2 Inputs

- **Environment:** A stable internet connection for both the client and server. The backend services and external Google Gemini AI API are operational.
- **Test Data:** A 3-minute audio file (.mp3) containing a clear, spoken-word lecture in English on a distinct topic (e.g., “The Basics of Photosynthesis”).
- **User State:** The user is logged into the web application.

3.3.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** After upload, the recording’s status should display as ‘Processing’. After a reasonable time, the status should update to ‘Completed’. Upon viewing the recording’s details, a structured summary relevant to the audio content must be displayed.
- **Pass Criteria:** The test passes if a non-empty, coherent summary is generated and correctly associated with the uploaded audio recording in the UI. The summary content must be contextually relevant to “The Basics of Photosynthesis”.

- **Fail Criteria:** The test fails if the status remains 'Processing' for an excessive duration (>15 minutes), the status changes to 'Failed', no summary is displayed for the completed recording, the summary is empty or nonsensical, or the application crashes.

3.3.4 Test Procedure

---

1. Log into the AudioScholar web application.
2. Navigate to the 'Upload Audio' page and upload the 3-minute test audio file.
3. Confirm the file appears in the 'My Recordings' list with a 'Processing' status.
4. Periodically refresh the recordings list.
5. Verify that within an acceptable timeframe (e.g., under 5 minutes), the status for the recording changes to 'Completed'.
6. Click on the completed recording to navigate to its details page.
7. Verify that a summary section is visible and contains structured text (e.g., headings, key points, glossary).
8. Read the generated summary to confirm its content is accurate and relevant to the topic of photosynthesis.

3.4 Case-4: User Login with Invalid Password

3.4.1 Purpose

To ensure the system's security and validation by verifying that it denies access when a user attempts to log in with a valid, registered email address but an incorrect password, and provides a clear, non-technical error message as per REQ-301.

3.4.2 Inputs

- **User Account:** A pre-existing user account has been registered with the email `validuser@test.com` and the password `CorrectPassword123!`.
- **Test Data:** Email: `validuser@test.com`, Password: `IncorrectPassword`.
- **Environment:** The mobile application is installed, and the device has an active internet connection.

3.4.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** The user should be prevented from logging in and should remain on the login screen. A specific error message indicating a credential mismatch should be displayed.
- **Pass Criteria:** The test passes if authentication is denied and a user-friendly error message (e.g., "Invalid email or password. Please try again.") is displayed prominently on the login screen.
- **Fail Criteria:** The test fails if the user is granted access to the application, the application crashes, a generic or technical error code (e.g., 'Authentication Error 401') is shown to the user, or no feedback is provided at all.

3.4.4 Test Procedure

1. Launch the AudioScholar mobile application.
2. If a user is logged in, log out to return to the login screen.
3. Enter `validuser@test.com` in the email input field.
4. Enter `IncorrectPassword` in the password input field.
5. Tap the 'Login' button.
6. Verify that the application does not navigate away from the login screen.
7. Verify that the specified user-friendly error message is displayed near the input fields or as a notification.

3.5 Case-5: Summarization with PowerPoint Contextual Enhancement

3.5.1 Purpose

To verify that when a user uploads both an audio file and an associated PowerPoint file, the backend correctly uses the PPT's content to enhance the AI summarization, resulting in a more accurate and context-rich summary (validating REQ-203).

3.5.2 Inputs

- **Environment:** User logged into the web interface with an active internet connection1. The backend and Google Gemini AI API are operational.
- **Test Data 1 (Audio):** A 2-minute audio file (.mp3) containing a general spoken lecture on "computer hardware" but *without* using the specific terms "RAM," "CPU," or "SSD."

- **Test Data 2 (PPT):** A PowerPoint file (.pptx) containing slides with the specific keywords: “RAM (Random Access Memory),” “CPU (Central Processing Unit),” and "SSD (Solid State Drive)"222.

3.5.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** The user successfully uploads both files3. After processing, the status updates to 'Completed’4. The resulting summary page should display the generated summary text and also indicate that a presentation is attached5. The summary text itself should contain specific keywords from the PowerPoint (e.g., “CPU,” “RAM,” “SSD”) that were *not* present in the audio-only file.
- **Pass Criteria:** The test passes if the generated summary is successfully displayed and verifiably contains the specific keywords (“CPU,” “RAM,” “SSD”) from the PowerPoint file, proving contextual enhancement occurred.
- **Fail Criteria:** The test fails if the upload of either file fails, the processing fails, the summary is not generated, or the summary text shows no evidence of using the keywords from the PowerPoint file.

3.5.4 Test Procedure

1. Log into the AudioScholar web application.
2. Navigate to the ‘Upload Recording’ page6.
3. In the “Audio File (Required)” section, select the 2-minute “computer hardware” audio file7.
4. In the “PowerPoint File (Optional)” section, select the test PPT file containing the keywords8.

5. Enter a title (e.g., “PPT Context Test”).

6. Click the ‘Upload Recording’ button9.

7. Navigate to the ‘My Recordings’ list and wait for the status to change from ‘Processing’ to 'Completed’10.

8. Click on the “PPT Context Test” recording to view its details11.

9. Verify the summary section is visible and that a “View PDF” (or similar) button for the presentation is present12.

10. Read the generated summary text and confirm that it includes the specific keywords “CPU,” “RAM,” and “SSD.”

3.6 Case-6: Freemium Feature Restriction (Background Recording)

3.6.1 Purpose

To verify that a user logged in with a “Basic” (Free) account is correctly prevented from using premium features, specifically “Background Recording,” as defined in the freemium model requirements (REQ-401).

3.6.2 Inputs

- **Hardware:** Android device (Android 7.0+).
- **Software/System State:** The AudioScholar mobile application is installed.

- **User Data:** A user is authenticated with an account known to be on the "Basic" (Free) tier.

3.6.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** In the 'Settings' menu, the "Background Recording" feature should be visibly marked as a premium feature and disabled (e.g., greyed out with a 'lock' icon)14. If the user starts a recording and then backgrounds the app (e.g., by pressing the Home button), the recording session must stop.
- **Pass Criteria:** The test passes if the feature is disabled in settings AND the recording session terminates when the application is moved to the background.
- **Fail Criteria:** The test fails if the user can enable the feature, or if the recording continues to run for more than a few seconds after the app is backgrounded.

3.6.4 Test Procedure

1. Launch the AudioScholar mobile application.
2. Log in using the "Basic" (Free) user account.
3. Navigate to the 'Settings' screen151515.
4. Locate the "Premium Features" or "Audio Recording Settings" section.
5. Verify that the "Background Recording" option is present but disabled and marked with an "Upgrade to Pro" label or icon16.
6. Navigate to the 'Record' screen17.
7. Tap the record button to begin recording. Confirm the timer starts18.

8. After 10 seconds, press the device's "Home" button, moving the app to the background.

9. Wait 10 seconds.

10. Re-open the AudioScholar application from the recent apps list.

11. Verify that the recording screen shows the recording is "Stopped" or "Paused," and the timer is not running.

3.7 Case-7: Successful User Login with Google OAuth 2.0

3.7.1 Purpose

To verify that a user can successfully authenticate and log into the mobile application using the “Login with Google” OAuth 2.0 provider, ensuring a seamless login flow (validating REQ-301).

3.7.2 Inputs

- **Hardware:** Android device (Android 7.0+) with an active internet connection.
- **Software/System State:** The AudioScholar mobile application is installed. The device has at least one Google account signed in via the Android settings. The user is currently logged out of the app.
- **Configuration:** The Google Sign-In SDK is correctly configured in the app.

3.7.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** Upon tapping the "Login with Google" button, the native Android account chooser dialog should appear. After selecting a valid Google account, the app should briefly show a loading indicator and then navigate the user to the main application dashboard (e.g., 'My Recordings' screen).
- **Pass Criteria:** The test passes if the user is successfully logged in, redirected to the app's main screen, and a valid session is created (as verified by visiting a protected area like 'My Recordings').

- **Fail Criteria:** The test fails if tapping the button does nothing, the Google login dialog fails to appear, selecting an account results in a user-facing error message (e.g., "Login Failed," "API Error"), or the user is not redirected to the main app dashboard after a successful-looking login.

3.7.4 Test Procedure

---

1. Launch the AudioScholar mobile application.
2. Ensure the user is on the main Login/Register screen.
3. Tap the "Login with Google" (or "G") button.
4. Verify that the native Android/Google account selection dialog appears, showing the Google accounts registered on the device.
5. Select a valid Google account from the list.
6. Observe the application for a loading indicator.
7. Verify that the application automatically navigates to the main dashboard or 'My Recordings' screen.
8. (Optional) Navigate to the 'Profile' or 'Settings' screen and verify that the user's email address matches the selected Google account.

3.8 Case-8: Manual Cloud Synchronization (Mobile to Web)

3.8.1 Purpose

To verify that locally created recordings on the mobile app can be successfully uploaded to the cloud (Nhost storage) via the manual "Sync Now" feature and then become accessible from the web interface, validating REQ-302.

3.8.2 Inputs

- **Hardware:** Android device (online) and workstation (online).
- **User State:** The user is logged into *both* the mobile app and web interface with the *same* user account.
- **Test Data:** A new, locally saved recording on the mobile device that does not yet exist in the cloud (e.g., "Sync Test File").
- **Configuration:** The mobile app's "Sync Mode" is set to "Manual" or a manual sync button is available.

3.8.3 Expected Outputs & Pass/Fail Criteria

- **Expected Outputs:** After initiating a manual sync on the mobile app, a progress indicator should appear, followed by a “Sync complete” message. Subsequently, refreshing the ‘My Recordings’ list on the web interface should show the “Sync Test File” as available.
- **Pass Criteria:** The test passes if the “Sync Test File” appears in the web interface’s recording list after the manual sync is completed on mobile.

- **Fail Criteria:** The test fails if the mobile app shows a sync error, the sync process never completes, or the recording fails to appear in the web interface list after a successful-looking sync.

3.8.4 Test Procedure

---

1. On the Android device, log in and create a new, 10-second local recording.
2. Save it with the title “Sync Test File.” (Ensure device is online for this test).
3. Navigate to the ‘Library’ and confirm “Sync Test File” is listed under 'Local'.
4. On the workstation, log into the web interface with the same account.
5. Navigate to the ‘My Recordings’ list and verify “Sync Test File” is *not* present.
6. On the mobile app, navigate to ‘Settings’ > 'Cloud Sync Settings'.
7. Ensure “Sync Mode” is set to “Manual” (or that a manual sync button is available).
8. Initiate the manual sync (e.g., from the ‘Library’ screen or a global ‘Sync Now’ button).
9. Verify a progress indicator is shown, followed by a “Sync complete” (or similar) message.
10. Return to the web interface on the workstation and refresh the ‘My Recordings’ page.
11. Verify that the recording titled “Sync Test File” is now visible in the list.

4. Appendix (Test Logs)

This appendix contains the execution logs, summary report, and incident reports for the "System Testing, Phase 1" cycle, conducted from October 25-26, 2025.

A.1 Test Execution Log

Test Case ID	Date Executed	Executed By	Build Version	Result (Pass/Fail)	Notes / Defect ID
TC-01	Oct 25, 2025	Claive Justin Barrientos	v1.0-beta (Mobile)	Pass	Offline recording works as expected even without internet connection.
TC-02	Oct 25, 2025	Claive Justin Barrientos	v1.0-beta (Web)	Pass	Client-side validation and message handling performed correctly

TC-03	Oct 25, 2025	Claive Justin Barrientos	v1.0-beta (Server/Web)	Pass	Summary matched lecture content; backend integration stable.
TC-04	Oct 26, 2025	Claive Justin Barrientos	v1.0-beta (Mobile)	Pass	Database authentication validation is successful.
TC-05	Oct 26, 2025	Claive Justin Barrientos	v1.0-beta (Web/Server)	Pass	Contextual enhancement confirmed; Powerpoint parsing integrated properly
TC-06	Oct 26, 2025	Claive Justin Barrientos	v1.0-beta (Mobile)	Pass	Freemium feature restriction for free-tier users is correctly implemented.

TC-07	Oct 26, 2025	Claive Justin Barrientos	v1.0-beta (Mobile)	Pass	Google OAuth login flow is secure and successful.
TC-08	Oct 26, 2025	Claive Justin Barrientos	v1.0-beta (Mobile/Web)	Pass	Manual cloud sync (Mobile to Web) is operational.

A.2 Test Summary Report

Project Name: AudioScholar

Test Cycle: System Testing, Phase 1

Build Version: v1.0 beta

Period: Oct 25, 2025, to Oct 26, 2025

- **Total Test Cases Planned:** [8]
- **Total Test Cases Executed:** [8]
- **Passed:** [8]
- **Failed:** [0]
- **Blocked:** [0]

Critical Defects Found: [0]

Overall Assessment:

The v1.0-beta build is stable, and all core functionalities defined in the test plan passed execution.

- **G1 (Recording/Management):** Offline recording and file upload validations (TC-01, TC-02) are working as specified.
- **G2 (AI/Summary):** The end-to-end summarization workflow (TC-03) and the PowerPoint contextual enhancement (TC-05) are both fully functional.
- **G3 (User/Sync):** Security and authentication flows for standard login (TC-04) and Google OAuth (TC-07) are secure and successful. Manual cloud sync (TC-08) is operational.
- **G4 (Freemium):** Feature restriction for free-tier users (TC-06) is correctly implemented.

No critical or high-severity defects were found. The build meets the exit criteria and is recommended to proceed to User Acceptance Testing (UAT).

A.3 Incident Report

As noted in the Test Summary Report (Section A.2), **0 Critical** and **0 Failed** test cases were recorded during this test cycle. Therefore, no formal incident reports were filed.

The template below is provided for use in future test cycles.

Incident ID: IR-[XXX]

Date Found: YYYY-MM-DD

Found By: [Tester Name]

Test Case ID: [TC-XXX-XX]

Severity: [Critical / High / Medium / Low]

Priority: [High / Medium / Low]

Summary:

[A one-sentence summary of the problem.]

Description:

[Detailed description of the issue, including what was observed.]

Steps to Reproduce:

1. [Step 1]

2. [Step 2]

3. [Step 3]

Expected Result:

[What the application should have done.]

Actual Result:

[What the application actually did.]

Environment:

- **Platform:** [e.g., Android 11, Chrome on Windows 10]
- **Build Version:** [e.g., v1.0-RC1]

Attachments:

[Screenshots, logs, etc.]

Assigned To: [Developer Name]

Status: [Open / In Progress / Resolved / Closed]